PostgreSQL query plan nodes

For longer descriptions and examples, the best source I've found is a series of blog posts at https://www.depesz.com/2013/04/16/explaining-the-unexplainable/

Scans & Materialization

Seq Scan

Linear search over all tuples, possibly applying a filter.

Index Scan, Index Scan Backward

A B-tree traversal on an index condition, followed by a linear scan on the B-tree index leaves for tuples, possibly applying a filter. The linear scan of the leaves is in ascending or descending order, respectively.

Index Only Scan

A B-tree traversal on an index condition, followed by a linear scan on the B-tree index leaves for tuples, possibly applying a filter. In this case, all the data needed for the query is in the index, so the table itself isn't accessed.

Bitmap Heap Scan

Fetches rows as specified by a previous Bitmap Index Scan. Rechecks the heap search key and scan apply an additional filter.

Bitmap Index Scan

Scan on a bitmap, creating an index of which tuples match the search key.

BitmapAnd, BitmapOr, BitmapNot

Combines bitmap indices produced by Bitmap Index Scan with logical AND, OR, and NOT, respectively.

Subquery Scan

Marks a scan for a subquery.

TID Scan

Scan for a tuple having the given tuple identifier (TID).

Function Scan

Scan on data generated by a built-in function, e.g., generate_series(), possibly applying a filter.

Values Scan

Scan on a constant set of data provided by a VALUES clause.

Materialize

Materialize rows by saving them to memory and/or disk.

CTE

Materialize rows, by saving them to memory and/or disk, of a Common Table Expression (i.e., in WITH) for later use in a CTE Scan.

CTE Scan

Linear search over the previously materialized tuples of a common table expression (i.e., WHERE), as produced by CTE, possibly applying a filter.

Joins

Nested Loop, Nested Loop Left Join, Nested Loop Ant Join

Nested loop join, possibly applying a join filter. Implements an inner, left, or anti join, respectively. Inputs for a right join are reversed, instead.

Hash

Creates a hash table for one side of a join.

Hash Join, Hash Left Join, Hash Right Join, Hash Full Join, Hash Anti Join

Hash join, matching on a join condition, using a previously created hash table, possibly applying a join filter. Implements an inner, left, right, outer, or anti join, respectively.

Merge Join, Merge Left Join, Merge Right Join, Merge Full Join, Merge Anti Join

Sort merge join, matching on a join condition. Both sides of the join must be previously sorted. Implements an inner, left, right, outer, or anti join, respectively.

Computation

Sort

Sort the input rows as needed by ORDER BY or before a MERGE JOIN. The sort algorithm can vary, and can be either in memory or using disk.

GroupAggregate

Aggregates previously sorted data for a GROUP BY clause. The result is sorted.

Hash Aggregate

Creates a temporary hash table to aggregate data for a GROUP BY clause or to eliminate duplicates, as with DISTINCT. The input doesn't need to be sorted, and the result is not sorted.

WindowAgg

Aggregates data for a window function.

Limit

Applies a LIMIT on the input rows. Aborts the underlying operations when the desired number of rows has been obtained.

Unique

Gets a unique value from the previously sorted input rows for DISTINCT.

Append

Combines multiple result sets into one so that a set operation can be applied to them.

SetOp

Apply a set operation to the previously sorted input data sets.

HashSetOp

Apply a set operation to the previously hashed input data sets.

Result

The result is a supplied constant value.

Other

Insert, Update, Delete

Apply table changes for an INSERT, UPDATE, or DELETE.

InitPlan

Indicates that part of the query must be executed before another part and that doesn't depend on another part of the query.

SubPlan

Indicates that part of the query must be executed before another part, but that does depend on the current row being scanned.